

Structural Arithmetic — Algebraic Model & Type Signatures

Goal: formalize a computational model where *relational structure* (dimensions/ratios) is distinct from *metric* (numeric magnitude), enabling exact symbolic manipulation of relations and deferred/arbitrary-precision evaluation of metrics.

1. High-level overview

A **Quantity** is a pair: $Q = (S, M)$ - S = Structural relation (dimension expression, term/graph).
- M = Metric mapping (scalar magnitude, possibly with unit metadata).

The system treats S as the canonical object for algebraic transformation; M is a functorial realization of S into numeric space ($\mathbb{R}$ or arbitrary precision field).

Operations on quantities split into two coordinated actions: 1. **Structural composition** — combine S terms algebraically (exact, symbolic).
2. **Metric composition** — combine M scalars using numeric arithmetic, only when allowed by structural compatibility.

2. Core algebraic types (informal)

```
-- Dimension expression: algebra over base dimensions
data DimExpr
  = Base String           -- "L", "T", "M", etc.
  | Pow DimExpr Integer   -- exponentiation, e.g. L^2
  | Mul DimExpr DimExpr   -- product of dimensional symbols
  | Div DimExpr DimExpr   -- quotient
  | One                   -- dimensionless

-- Quantity: structural expression + metric scalar
data Quantity scalar = Quantity
  { structure :: DimExpr
  , metric    :: scalar  -- e.g. Rational, BigDecimal, Interval, Symbolic
  }
```

Python (dataclass) analog:

```
from dataclasses import dataclass

@dataclass(frozen=True)
class DimExpr:
    # recursive representation; see text for constructors
```

```

pass

@dataclass
class Quantity:
    structure: DimExpr
    metric: Any # rational, Decimal, symbolic, etc.

```

3. Algebraic laws and constraints

- **Add/Subtract** only allowed when `structure` is equal (isomorphic after simplification).
- `Q1 + Q2` requires `S1 == S2`. Result `S = S1`, `M = metricOp(M1, M2)`.
- **Multiply/Divide** compose structures by algebraic operations:
 - `Q1 * Q2` -> `S = Mul(S1, S2)`, `M = M1 * M2`.
 - `Q1 / Q2` -> `S = Div(S1, S2)`, `M = M1 / M2`.
- **Exponentiation**: `Q ^ n` -> `S = Pow(S, n)`, `M = M^n`.
- **Structural simplification**: rewrite rules to canonicalize `DimExpr` (commute, combine powers, remove `One`).
- **Metric invariants**: The numeric metric layer must respect structural transformations; e.g., factoring a shared irrational symbol should not change the rational coefficient semantics.

4. Category-theoretic framing (brief)

- Objects: `DimExpr` (or the free abelian group generated by base dimensions).
- Morphisms: scalings/transformations between same object type (i.e., dimension-preserving maps).
- The numeric evaluation is a **monoidal functor** `F : (DimCategory) -> (Vect/Monoid over  $\mathbb{R}$ )` which assigns to each `S` a numeric representation space and to each structural composition a multiplication of scalars.

Switching metric systems corresponds to composing `F` with another change-of-basis natural transformation.

5. Representation choices

5.1 DimExpr canonicalization

- Represent `DimExpr` as a map `Map<BaseDim, Integer>` (exponent map).
e.g. `L^1 * T^-2` -> `{L:1, T:-2}`.
- This makes equality, multiplication, inversion trivial.

5.2 Metric types

- **Exact rational**: `Fraction` or big-int numerator/denominator — used for rational scalings.

- **Symbolic coefficient:** allow algebraic symbols (π , $\sqrt{2}$) as atomic metric tokens with identity. These can be factored/collected during consolidation.
- **Arbitrary precision:** `BigDecimal`, `BigFloat`, or interval arithmetic for error bounds.
- **Mixed:** a `MetricExpr` that is itself a product of exact rationals and symbolic atoms and numeric approximations.

Example metric representation (normalized):

```
MetricExpr = Rational * [] [SymbolicAtom^k] * Approximate(BigFloat)
```

6. Evaluation pipeline (operational semantics)

1. **Parse/Construct:** build `Quantity` from inputs (literal `10 m/s` $\rightarrow$ `structure={L:1,T:-1}`, `metric=10`).
2. **Symbolic composition:** perform algebraic ops on `structure` only; keep metrics attached but don't evaluate approximations.
3. **Consolidation / Factorization:** collect like symbolic atoms in metrics and factor common bases; optionally rewrite structure using known identities (e.g., treat π as atomic and factor π out of sums where possible — only valid in multiplicative contexts).
4. **Compatibility checks:** for additions/subtractions verify structural equality; otherwise produce symbolic residuals or throw dimension error.
5. **Final numeric realization:** apply a metric functor `F` to transform metric expressions to arbitrary-precision numerics — optionally substituting numeric values for symbolic atoms (e.g., $\pi \rightarrow \text{BigFloat}(3.1415\dots)$).
6. **Post-evaluation rounding/interval propagation:** produce final approximations with guaranteed error bounds.

7. Examples

1) Velocity and acceleration

```
v = 10 m/s => (S={L:1,T:-1}, M=10)
t = 2 s      => (S={T:1}, M=2)

a = v / t    => S = {L:1,T:-2}, M = 10 / 2 = 5
```

Structural operations ensure `a` carries dimensional meaning independent of `m/s` vs `cm/min`.

2) Symbolic consolidation

```
q1 = pi * m => S={L:1}, M = pi
q2 = 2 * m  => S={L:1}, M = 2

q = q1 + q2 => allowed since S equal => M = pi + 2 (kept symbolic)
```

Only at final realization do we replace π with an approximate numeric value.

8. API / Type signatures (practical)

```
# Python-like API sketch
class DimExpr:
    @staticmethod
    def from_unit_string(s: str) -> DimExpr: ...
    def canonical(self) -> Dict[str,int]: ...

class MetricExpr:
    # supports rational, symbolic atoms, approximate component
    def factor(self): ...
    def approximate(self, substitutions: Dict[str, Decimal], prec: int): ...

class Quantity:
    def __add__(self, other: Quantity) -> Quantity: ...
    def __mul__(self, other: Quantity) -> Quantity: ...
    def evaluate(self, substitutions: Dict[str, Numeric], prec: int) ->
    Quantity
```

`evaluate()` applies a metric functor: resolves symbolic atoms, computes arbitrary precision numeric result, returns a new `Quantity` with `structure` preserved but `metric` now numeric.

9. Consolidation & collection algorithm (sketch)

- Maintain metrics as multiset of atomic tokens: `{(atom, exponent): coefficient}` where `atom` may be π , $\sqrt{2}$, rational prime factors, or an `Approx` node.
- When multiplying/dividing, merge exponent maps.
- When adding, attempt to extract a common factor across terms by GCD-like algorithm on metric multisets; if impossible, keep a symbolic sum (exact representation) until substitution.

This avoids premature numeric cancellation and preserves algebraic structure for later manipulation.

10. Implementation considerations

- **Performance tradeoff:** symbolic tracking increases memory/time but reduces numeric error and enables richer reasoning.
 - **Interoperability:** provide clear conversion boundaries (`Quantity` -> plain float) and tools to pin metrics globally (e.g., unit system context).
 - **Error semantics:** use interval arithmetic or affine arithmetic to track numeric error during final realization.
 - **Extensibility:** allow custom symbolic atoms and algebraic relations (e.g., treat `g` or `c` as named constants with known substitution values).
-

11. Roadmap / Next steps

- Implement `DimExpr` canonical map and `MetricExpr` multiset.
 - Build a small interpreter that defers numeric evaluation and exposes `consolidate()` and `evaluate()` hooks.
 - Add rewrite rules for common algebraic simplifications and user-provided identities.
 - Integrate arbitrary-precision backends and interval arithmetic for robust final evaluation.
-

If you want, I can now: - produce a minimal reference implementation in Python (dataclasses + basic ops), or - write a concise Haskell module that uses the type system to enforce dimensional constraints, or - sketch how this would integrate into your Bootstrap IDE graph model.

Which do you want next?